Big Data Analytics (BIS701) - Programs 2 to 9

Program 2: Matrix Multiplication

```
from collections import defaultdict
A=[[1,2,3],[4,5,6]]; B=[[7,8],[9,10],[11,12]]
g=defaultdict(int)
for i in range(len(A)):
    for j in range(len(B[0])):
        for k in range(len(B)):
            g[(i,j)]+=A[i][k]*B[k][j]   # map + reduce(sum)
for i in range(len(A)): print([g[(i,j)] for j in range(len(B[0]))])
```

Output: [58, 64] and [139, 154]

Program 3: Weather Data

```
from collections import defaultdict, Counter
data="""2024-04-01,Sunny
2024-04-01,Rainy
2024-04-01,Sunny
2024-04-02,Cloudy
2024-04-02,Cloudy
2024-04-02,Rainy
2024-04-03,Sunny"""
d=defaultdict(Counter)
for l in data.split():
    dt,c=l.split(','); d[dt][c]+=1
for dt,c in d.items(): print(f"Weather on {dt}: {c.most_common(1)[0][0]}")
```

Program 4: MovieLens Tags

```
from collections import defaultdict
movies={1:'Toy Story',2:'Jumanji',3:'Grumpier Old Men'}
tags=[(1,'funny'),(1,'animation'),(2,'adventure'),(2,'game'),(1,'funny'),(3,'romantic')]
d=defaultdict(set)
for m,t in tags: d[m].add(t)
for m,t in d.items(): print(f"Movie: {movies[m]}, Tags: {', '.join(sorted(t))}")
```

Program 5: MongoDB

```
use exp5
db.c.insertMany(Array.from({length:10000},(_,i)=>({
  name:"Person"+i,age:Math.floor(Math.random()*100),city:["Delhi","Chicago","Houston"][i%3]})))
db.c.countDocuments()
db.c.find().sort({age:-1}).skip(5).limit(5)
db.c.aggregate([{$group:{_id:"$city",avgAge:{$avg:"$age"},n:{$sum:1}}}])
```

Program 6: Pig Latin

Run with: pig -x local emp.pig (put each department on its own line in departments.txt)

```
e = LOAD 'employees.txt' USING PigStorage(',') AS (id:int,name:chararray,age:int,dept:int,sal:int);
d = LOAD 'departments.txt' USING PigStorage(',') AS (dept:int,dname:chararray);
f = FILTER e BY age>28;
p = FOREACH f GENERATE id,name,sal;
s = ORDER p BY sal DESC;
g = GROUP e BY dept;
j = JOIN e BY dept, d BY dept;
DUMP s; DUMP g; DUMP j;
```

Program 7: Hive

```
CREATE DATABASE db; USE db;
ALTER DATABASE db SET DBPROPERTIES('Owner'='Admin');
CREATE TABLE emp(id INT,name STRING,dept INT,sal FLOAT)
  ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
ALTER TABLE emp ADD COLUMNS(email STRING);
ALTER TABLE emp RENAME TO emp2;
CREATE VIEW v AS SELECT id,name FROM emp2 WHERE sal>50000;
ALTER VIEW v AS SELECT id,name,sal FROM emp2 WHERE sal>60000;
DROP VIEW v;
ADD JAR /home/cloudera/udf.jar;
CREATE FUNCTION to_upper AS 'com.example.hiveudf.ToUpperUDF';
DROP FUNCTION to_upper;
CREATE INDEX i ON TABLE emp2(dept) AS 'COMPACT' WITH DEFERRED REBUILD;
```

```
ALTER INDEX i ON emp2 REBUILD;
DROP INDEX i ON emp2;
DROP TABLE emp2;
DROP DATABASE db CASCADE;
```

Note: indexes only work in Hive versions below 3.0, which Cloudera's CDH has.

Program 8: Word Count

Hadoop (Java)

```
import java.io.IOException;
import java.util.StringTokenizer;
import org.apache.hadoop.conf.Configuration;
import org.apache.hadoop.fs.Path;
import org.apache.hadoop.io.*;
import org.apache.hadoop.mapreduce.*;
import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;
import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;

public class WordCount {
  public static class M extends Mapper<Object,Text,Text,IntWritable>{
    public void map(Object k,Text v,Context c) throws IOException,InterruptedException{
      StringTokenizer t=new StringTokenizer(v.toString());
      while(t.hasMoreTokens()) c.write(new Text(t.nextToken()),new IntWritable(1));
    }
  }
  public static class R extends Reducer<Text,IntWritable,Text,IntWritable>{
    public void reduce(Text k,Iterable<IntWritable> v,Context c)
        throws IOException,InterruptedException{
      int s=0; for(IntWritable i:v) s+=i.get();
      c.write(k,new IntWritable(s));
    }
  }
  public static void main(String[] a) throws Exception{
    Job j=Job.getInstance(new Configuration(),"wc");
    j.setJarByClass(WordCount.class);
    j.setMapperClass(M.class); j.setCombinerClass(R.class); j.setReducerClass(R.class);
    j.setOutputKeyClass(Text.class); j.setOutputValueClass(IntWritable.class);
    FileInputFormat.addInputPath(j,new Path(a[0]));
    FileOutputFormat.setOutputPath(j,new Path(a[1]));
    System.exit(j.waitForCompletion(true)?0:1);
  }
}
```

```
javac -classpath `hadoop classpath` -d . WordCount.java
jar cf wc.jar WordCount*.class
hadoop jar wc.jar WordCount /user/cloudera/input /user/cloudera/output
hdfs dfs -cat /user/cloudera/output/part-r-00000
```

Spark (PySpark)

```
from pyspark import SparkContext
sc=SparkContext("local","wc")
r=(sc.textFile("sample.txt").flatMap(str.split)
   .map(lambda w:(w,1)).reduceByKey(lambda a,b:a+b))
for w,cnt in r.collect(): print(w,cnt)
```

Run with: spark-submit wordcount.py

Program 9: CDH + HUE

```
hdfs dfs -mkdir -p /user/cloudera/data
hdfs dfs -put employees.csv /user/cloudera/data/
```

Open http://localhost:8888 (cloudera/cloudera), go to Query Editors > Hive, and run:

```
CREATE DATABASE IF NOT EXISTS company; USE company;
CREATE TABLE employees(id INT,name STRING,department STRING,salary INT)
  ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
LOAD DATA INPATH '/user/cloudera/data/employees.csv' INTO TABLE employees;
SELECT department,MAX(salary) FROM employees GROUP BY department;
SELECT * FROM employees WHERE salary>65000;
SELECT department,COUNT(*) FROM employees GROUP BY department;
```

For the report, click Export (CSV/Excel) on the results, or use Dashboard > Add Widget for a chart.